

とまれない

PORTFOLIO Game Programmer



跳弾

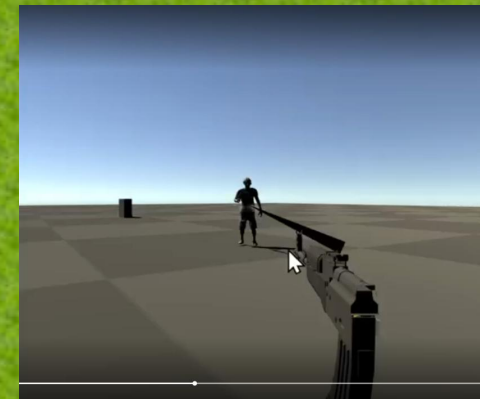
【2Dシューティングゲーム】

とまれない

【2Dアクションゲーム】

■使用技術

- C++ • C# • Visual Studio
- Unity



Unity作品 (制作途中)

【3Dアクションシューティングゲーム】

Game Programmer

吉岡 市之介

CONTENTS

P. 02~ PROFILE

P. 03~ とまれない
2D横スクロールシミュレーション

P. 06~ 跳弾
2Dシューティングゲーム

P. 09~ Unity作品(制作途中)
3Dアクションシューティングゲーム

P. 12~ 学んだこと

P. 13 プレイ映像URL/Git Hub URL

P. 01

PROFILE

■氏名：吉岡 市之介
(よしおか いちのすけ)

■使用言語
・C++ ・C#

■開発環境
・Visual Studio ・Unity

■取得資格
・基本情報技術者試験

■志望職種
・ゲームプログラマー

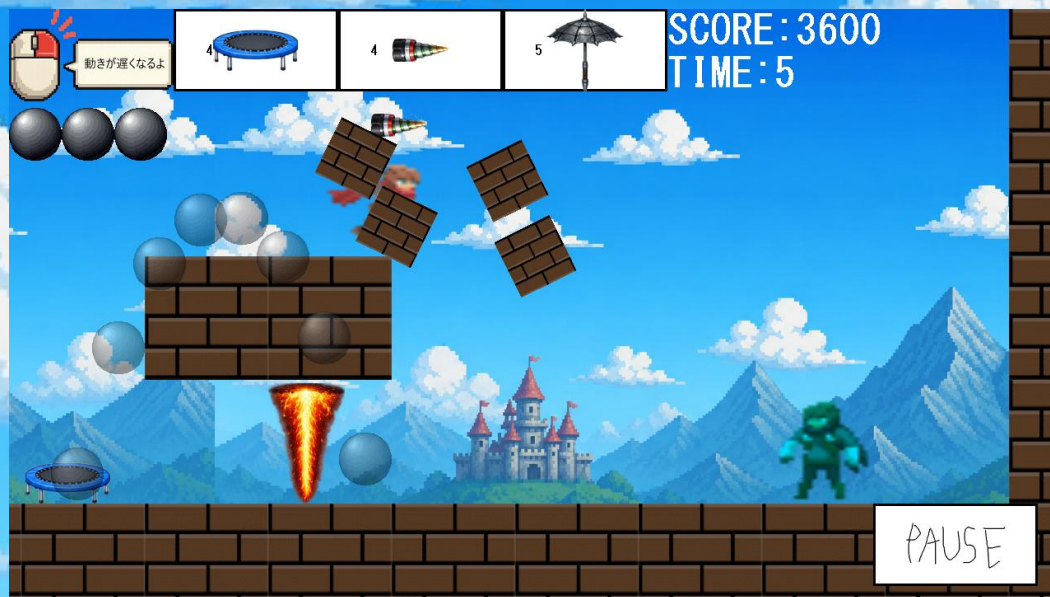
■得意分野
・ゲームプログラミング
・オブジェクト指向
・保守性・拡張性を意識した設計

■個人制作
・とまれない・跳弾
・Unity作品

■自己PR

私は、保守性・拡張性を意識しながらゲーム開発に取り組むことを大切にしています。個人制作ではC++を用いて2Dゲームを開発、Unityを用いて3Dゲームを開発し、プレイヤーの挙動や敵AI、当たり判定などの実装を担当しました。仕様変更にも対応しやすい設計を心掛け、クラスの責務を整理し共通処理をまとめるなど改善を重ねました。今後も技術力を磨き、チームで高品質なゲーム開発に貢献できるプログラマーを目指します。

とまれない 2D横スクロールシミュレーション



※ゲームメイン

■ジャンル
2D横スクロールシミュレーション

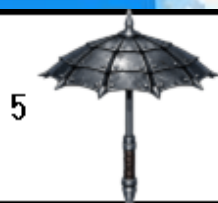
■制作期間
2026年3月~ 2026年6月
3ヶ月（週に3回7時間）

■開発環境
C++
Visual Studio

■制作形態
個人製作

■コンセプト
プレイヤーが状況に応じてアイテムを自由に選択し、それぞれの特性を活かしてステージを攻略する。

ゲームシステム



■ アイテム選択

プレイヤーが自由に
アイテムを選択設置可能

■ 敵

敵との接触でライフが減少。上か
ら踏みつけることができる

■ スローモード

右クリックで動きが遅くなる



動きが遅くなるよ

■ ギミック

もともとステージに設置されて
いるギミック

■ クリア条件 (ギミック)

ゴールに接触でゲームクリア



■ ゲームオーバー条件

ライフが0になる



■ 自キャラ

操作不可で、壁に接触するまで前に走り続ける。アイテムなど
に接触することでジャンプなどのアクションを行う

プログラム設計・工夫した点

```
└─ character_manager
   └─ character
      ├── ground_enemy
      ├── player
      ├── ++ character.cpp
      └─ character.h
   ├── character_id.h
   ├── ++ character_manager.cpp
   └─ character_manager.h
```

```
character
├─ player
└─ ground_enemy
```

```
└─ item_object_manager
   └─ item_object
      ├── drill
      ├── trampoline
      ├── umbrella
      ├── ++ item_object.cpp
      └─ item_object.h
   ├── item_object_id.h
   ├── ++ item_object_manager.cpp
   └─ item_object_manager.h
```

```
item_object
├─ trampoline
├─ drill
└─ umbrella
```

```
└─ stage_gimmick_manager
   └─ stage_gimmick
      ├── check_point
      ├── fire_gimmick
      ├── goal_gimmick
      ├── spike_gimmick
      ├── ++ stage_gimmick.cpp
      ├── stage_gimmick.h
      ├── stage_gimmick_direction.h
      ├── stage_gimmick_id.h
      ├── ++ stage_gimmick_manager.cpp
      └─ stage_gimmick_manager.h
```

```
stage_gimmick
├─ spike
├─ fire
├─ goal
└─ check_point
```

```
└─ collision
   ├── ++ collision.cpp
   └─ collision.h
```

■設計で工夫した点

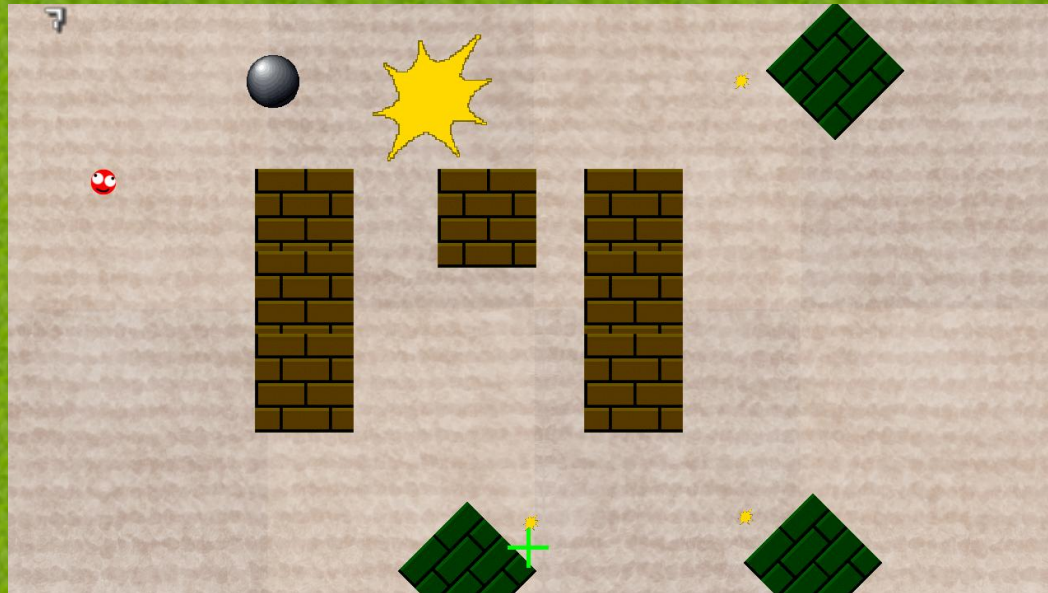
- 役割ごとにManagerを分割し、**責務を明確化**

- 新しいアイテムやギミックは派生クラスを追加するだけで**拡張できる構成**にした

- 基底クラスを作成し、**継承**によって**共通処理を集約**

- collisionクラスを作ることによって判定の処理を**簡略化**する

跳弾 2Dシューティングゲーム



※ゲームメイン

■ジャンル
2Dシューティング

■制作期間
2025年10月~2026年2月
4ヶ月（週に3回7時間）

■開発環境
C++
Visual Studio

■制作形態
個人制作

■コンセプト
敵との位置関係や跳弾角度を考えながら攻略する。

ゲームシステム



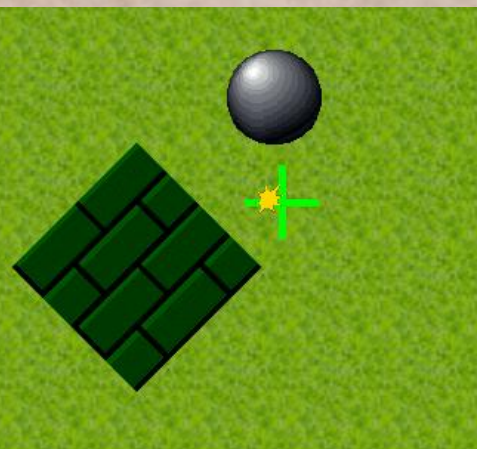
■弾

跳ね返る回数がステージによって設定されてる。



■敵

弾が接触すると撃破



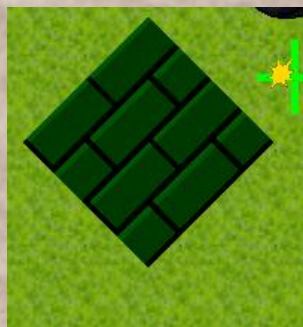
■跳弾

弾が接触したブロックや壁の角度に対して反射



■クリア条件

敵を全滅させる



■移動ブロック

自分で設置して回転できる



■固定ブロック

各ステージに最初から設置されてる

プログラム設計・工夫した点

```
void CBullet::HitMove(IBlock* block)
{
    aqua::CVector2 object_position = block->GetCenterPosition();
    float object_size = block->GetSize();
    float object_angle = block->GetAngle();

    // オブジェクトからボールまでのベクトルを求めて
    // オブジェクト空間にローカライズ
    aqua::CVector2 v = GetCenterPosition() - object_position;

    //オブジェクトに対してのベクトルの角度を求める
    float angle = aqua::RadToDeg(atan2(v.y, v.x));
    float rad = aqua::DegToRad(angle);
    aqua::CVector2 vector;
    vector.x = cos(rad);
    vector.y = sin(rad);

    //オブジェクトの角度に応じてベクトルを回転させる
    vector = aqua::CVector2::RotationZ(vector, -aqua::DegToRad(object_angle));
    v = aqua::CVector2::RotationZ(v, -aqua::DegToRad(object_angle));

    //オブジェクトの角度に応じて加速度を回転させる
    aqua::CVector2 localized_velocity = aqua::CVector2::RotationZ(m_Velocity, -aqua::DegToRad(object_angle));

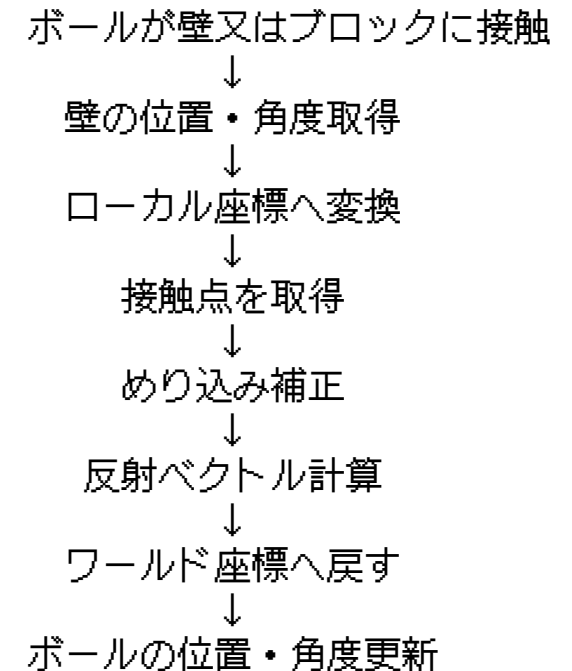
    //オブジェクトの当たり位置を求める
    aqua::CVector2 near_position = GetNearPosition(v, vector, object_angle, object_size);

    //跳ね返った後のVelocityの更新
    switch (block->GetDirectionLocal(near_position)) {
        //オブジェクトに埋まらない対策
        v = BackPosition(vector) + (m_Radius*5.0f)*vector;
        //角度に応じた加速度に変換
        m_Velocity = aqua::CVector2::RotationZ(localized_velocity, aqua::DegToRad(object_angle));
        //跳ね返った後の位置
        m_Position = object_position + aqua::CVector2::RotationZ(v, aqua::DegToRad(object_angle))-aqua::CVector2(m_Radius, m_Radius);
        //跳ね返った後の角度
        m_Angle = aqua::RadToDeg(atan2(m_Velocity.y, m_Velocity.x));
    }
}
```

※跳弾 (弾の跳ね返り) の処理

跳弾の処理フロー

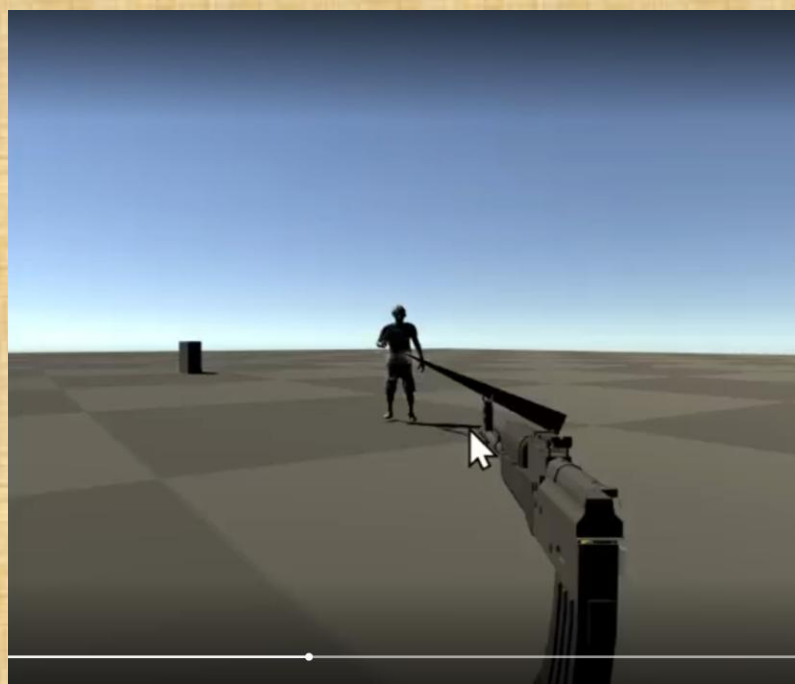
跳弾処理



■工夫した点

跳弾がゲームの核となるため、反射処理を関数としてまとめ、複数の壁でも同じ処理を利用できるように設計しました。また、弾・壁・敵の役割を分離し、機能追加や修正がしやすい構成を意識しました。

Unity作品 (制作途中) 3D横シューティングアクション



※ゲームメイン

■コンセプト

スキルを駆使して敵を駆逐する

■ジャンル

3D横シューティングアクション

■制作期間

2026年7月~ 2026年8月

1ヶ月 (週に3回7時間)

■開発環境

C#

Unity

■制作形態

個人製作

■素材

フリー3Dモデルやエフェクト、
モーションを使っています。

ゲームシステム

■自キャラ

歩き走りジャンプなどの**基本的な操作**ができる

■メインスキル・サブスキル

二つのスキルをつけることができる。現在は、入力方向に**瞬間移動するスキル**と、**相手の弾を跳ね返すスキル**を実装済み

■銃

弾を発射することが可能。**使う弾を選ぶことができる**。遅い弾と、射撃した瞬間に当たる弾を実装済み

■敵キャラ

ターゲットを見つけると近づき**射程圏内に入ると射撃**してくる

Unityでの実装経験

■実装経験

- PlayerController
- Input System
- Rigidbodyによる物理制御
- NavMeshによる敵AI
- Raycastによる視認・当たり判定
- Animatorによるアニメーション制御
- 弾の発射処理
- TrailRendererによる弾道表現
- Particle System
- LayerMask
- Coroutine

学んだこと

■学んだこと

- 保守性
- 拡張性
- 責務分割
- 継承

■今後挑戦したいこと

- 設計力向上
- AI

ゲーム制作を通して、保守性・拡張性を考慮した設計の重要性を学びました。今後も新しい技術を積極的に学び、より高品質なゲーム開発ができるプログラマーを目指します。

プレイ映像URL Git Hub URL

ありがとうございました。
プレイ動画はこちらからご覧いただけます。

とまれない(約50秒)

https://youtu.be/fFixunml_2Q



跳弾(約1分)

<https://youtu.be/5ANTCj-mXgc>



Git Hub URL

[Issues · 1chi11/game-project](https://github.com/1chi11/game-project/issues)